

Lecture 12 Relational Database Design

MVD

United International College

Outline

- Features of Good Relational Design
- Atomic Domains and First Normal Form
- Functional Dependency Theory
- BCNF
- 3rd Normal Form
- Multivalued Dependencies and Decomposition
- Database-Design Process

Multivalued Dependencies

- Suppose we record ISBNs of books borrowed by students, and their phone numbers.

$$\text{borrow} = \{\underline{id}, ISBN\}$$
$$\text{contact} = \{\underline{id}, \text{phone}\}$$

- Please note the keys. “*phone*” is multivalued to each student.
- Thus, the candidate keys consist of all attributes, and there is no functional dependencies for the schemas.
- If we combine these schemas and get

$$\text{bor_con} = \{\underline{id}, ISBN, \text{phone}\}$$

- This relation is in BCNF. Why?
 - No functional dependencies after combine.

Multivalued Dependencies

- Example data:

<i>ID</i>	<i>ISBN</i>
99999	978-0465026562
99999	978-1439851296

<i>ID</i>	<i>phone_number</i>
99999	512-555-1234
99999	512-555-4321

- After combine, we see redundancies.

<i>ID</i>	<i>ISBN</i>	<i>phone_number</i>
99999	978-0465026562	512-555-1234
99999	978-0465026562	512-555-4321
99999	978-1439851296	512-555-4321
99999	978-1439851296	512-555-1234

- Thus, we need another normal form to handle the multivalued cases.

Multivalued Dependencies

- Let R be a relation schema and let $\alpha \subseteq R$ and $\beta \subseteq R$. The **multivalued dependency**

$$\alpha \twoheadrightarrow \beta$$

holds on R if in any legal relation $r(R)$, for all pairs for tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, there exist tuples t_3 and t_4 in r such that:

- $t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha]$
- $t_3[\beta] = t_1[\beta]$
- $t_3[R \setminus (\alpha \cup \beta)] = t_2[R \setminus (\alpha \cup \beta)]$
- $t_4[\beta] = t_2[\beta]$
- $t_4[R \setminus (\alpha \cup \beta)] = t_1[R \setminus (\alpha \cup \beta)]$

MVD

- A tabular representation of $\alpha \twoheadrightarrow \beta$

	α	β	$R \setminus (\alpha \cup \beta)$
t_1	$a_1, \dots, a_i$	$a_{i+1}, \dots, a_j$	$a_{j+1}, \dots, a_n$
t_2	$a_1, \dots, a_i$	$b_{i+1}, \dots, b_j$	$b_{j+1}, \dots, b_n$
t_3	$a_1, \dots, a_i$	$a_{i+1}, \dots, a_j$	$b_{j+1}, \dots, b_n$
t_4	$a_1, \dots, a_i$	$b_{i+1}, \dots, b_j$	$a_{j+1}, \dots, a_n$

Example

- Let R be a relation schema with a set of attributes that are partitioned into 3 nonempty subsets.

Y, Z, W

- We say that $Y \twoheadrightarrow Z$ (Y **multidetermines** Z) if and only if for all possible relations $r(R)$

if

$$\langle y_1 z_1 w_1 \rangle \in r \text{ and } \langle y_1 z_2 w_2 \rangle \in r$$

then

$$\langle y_1 z_1 w_2 \rangle \in r \text{ and } \langle y_1 z_2 w_1 \rangle \in r$$

- Note that since the behavior of Z and W are identical it follows that

$$Y \twoheadrightarrow Z \text{ if } Y \twoheadrightarrow W$$

Example

- In the previous example on P.g. 4:

$$ID \twoheadrightarrow ISBN$$

$$ID \twoheadrightarrow phone$$

- The above formal definition is supposed to formalize the notion that given a particular value of $Y = \{ID\}$, which has associated with a set of values of $Z = \{ISBN\}$ and $W = \{phone\}$, and these two sets are in some sense independent of each other.
- Note:
 - If $Y \rightarrow Z$ then $Y \twoheadrightarrow Z$.
 - Indeed, we have (in above notation) $Z_1 = Z_2$

Use of Multivalued Dependencies

- We use multivalued dependencies in two ways:
 1. To test relations to **determine** whether they are legal under a given set of functional and multivalued dependencies.
 2. To specify **constraints** on the set of legal relations. We shall thus concern ourselves *only* with relations that satisfy a given set of functional and multivalued dependencies.
- If a relation r fails to satisfy a given multivalued dependency, we can construct a relations r' that does satisfy the multivalued dependency by adding tuples to r .

Theory of MVDs

- From the definition of multivalued dependency, we can derive the following rule:

- If $\alpha \rightarrow \beta$, then $\alpha \twoheadrightarrow \beta$

That is, every functional dependency is also a multivalued dependency.

- The **closure** D^+ of D is the set of all functional and multivalued dependencies logically implied by D .
 - We can compute D^+ from D , using the formal definitions of functional dependencies and multivalued dependencies.
 - We can manage with such reasoning for very simple multivalued dependencies, which seem to be most common in practice.
 - For complex dependencies, it is better to reason about sets of dependencies using a system of inference rules (see Appendix C in the textbook).

Fourth Normal Form

- The fourth normal form is the multivalued version of BCNF.
- A relation schema R is in **4NF** with respect to a set D of functional and multivalued dependencies if for all multivalued dependencies in D^+ of the form $\alpha \twoheadrightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following hold:
 - $\alpha \twoheadrightarrow \beta$ is trivial (i.e., $\beta \subseteq \alpha$ or $\alpha \cup \beta = R$)
 - α is a superkey for schema R .
- If a relation is in 4NF it is in BCNF.
- The schema on page 4 is not in 4NF.
$$\text{bor_con} = \{\text{id}, \text{ISBN}, \underline{\text{phone}}\}$$
- Because $\text{id} \twoheadrightarrow \text{ISBN}$ and $\text{id} \twoheadrightarrow \text{phone}$, but id is not a superkey.

Restriction of Multivalued Dependencies

- The restriction of D to R_i is the set D_i consisting of
 - all functional dependencies in D^+ that include only attributes of R_i and
 - all multivalued dependencies of the form
$$\alpha \twoheadrightarrow (\beta \cap R_i)$$
where $\alpha \subseteq R_i$ and $\alpha \twoheadrightarrow \beta$ is in D^+ .

4NF Decomposition Algorithm

Algorithm 4NF Decomposition

1. $result = \{R\}$
 2. compute D^+
 3. Let D_i denote the restriction of D^+ to R_i
 4. **while** $\alpha \twoheadrightarrow \beta \in D_i$ to R_i violates 4NF **do**
 5. $result = \{result \setminus R_i\} \cup \{R_i \setminus \beta\} \cup \{\alpha \cup \beta\}$
 6. **end while**
 7. **return** $result$
-

Note:

- 4NF decomposition is very close to BCNF decomposition.
- The set of multivalued dependencies is calculated in the same way as BCNF decomposition.
- Each R_i is in 4NF, and decomposition is lossless-join.

Example

- $R = \{A, B, C, G, H, I\}$, $D = \{A \twoheadrightarrow B, B \twoheadrightarrow HI, CG \twoheadrightarrow H\}$
- R is not in 4NF since $A \twoheadrightarrow B$ and A is not a superkey for R

- Decomposition R

$$R_1 = \{A, B\} \qquad D_1 = \{A \twoheadrightarrow B\} \qquad (\text{in 4NF})$$

$$R_2 = \{A, C, G, H, I\} \qquad D_2 = \{A \twoheadrightarrow HI, CG \twoheadrightarrow H\} \qquad (\text{not in 4NF})$$

- Decomposition R_2

$$R_3 = \{C, G, H\} \qquad D_3 = \{CG \twoheadrightarrow H\} \qquad (\text{in 4NF})$$

$$R_4 = \{A, C, G, I\} \qquad D_4 = \{A \twoheadrightarrow I\} \qquad (\text{not in 4NF})$$

- F_4 is the restriction of F to R_4 .

- Decomposition R_4

$$R_5 = \{A, I\} \qquad D_5 = \{A \twoheadrightarrow I\} \qquad (\text{in 4NF})$$

$$R_6 = \{A, C, G\} \qquad D_6 = \{\} \qquad (\text{in 4NF})$$

- The result is $\{R_1, R_3, R_5, R_6\}$

Further Normal Forms

- **Join dependencies** generalize multivalued dependencies.
- It leads to **project-join normal form (PJNF)** (also called **fifth normal form**).
- A class of even more general constraints, leads to a normal form called **domain-key normal form**.
- These generalized constraints are hard to reason with, and no set of sound and complete set of inference rules.
- Hence, those normal forms are rarely used.

Outline

- Features of Good Relational Design
- Atomic Domains and First Normal Form
- Functional Dependency Theory
- BCNF
- 3rd Normal Form
- Multivalued Dependencies and Decomposition
- **Database-Design Process**

Design Goals

- Goal for a relational database design is:
 - BCNF
 - lossless join, and
 - dependency preservation.
- If we cannot achieve this, we accept one of
 - lack of dependency preservation or
 - redundancy due to use of 3NF
- Interestingly, SQL does not provide a direct way of specifying functional dependencies other than superkeys.
- We can specify FDs using assertions, but they are expensive to test.

Overall Database Design Process

- For schema R
 - R could have been generated when converting E-R diagram to a set of tables.
 - R could have been a single relation containing ***all*** attributes that are of interest (called **universal relation**).
 - Normalization breaks R into smaller relations.
 - R could have been the result of some ad-hoc design of relations, which we then test/convert to normal form.

ER Model and Normalization

- When an E-R diagram is carefully designed, identifying all entities correctly, the tables generated from the E-R diagram should not need further normalization.
- However, in a real (imperfect) design, there can be functional dependencies from non-key attributes of an entity to other attributes of the entity.
 - Example: an *employee* entity with attributes
$$\{department_number, department_address\}$$
and a functional dependency set
$$\{department_number \rightarrow department_address\}$$
 - A good design would have made department an entity.
- Functional dependencies from non-key attributes of a relationship set possible, but rare - most relationships are binary.

End of Lecture 12